DAY 25

Reliable processing.
Consistent outcomes.
Resilient systems.

INBOX PATTERN, EXACTLY-ONCE EFFECT, AND CONSUMER RELIABILITY

At-least-once delivery from brokers.
Exactly-once effect in your business.

THE GOAL

Ensure each logical message affects business state once, even when delivery happens multiple times.

WHY INBOX PATTERN EXISTS

- Brokers provide at-least-once delivery, not exactly-once processing.
- In-memory dedup can fail on restarts and lose seen-message history.
- Inbox pattern stores processed message IDs durably at the consumer boundary.
- It guarantees exactly-once business effect, even when delivery is repeated.

HOW INBOX PATTERN WORKS (HIGH LEVEL)

BROKER → CONSUMER → INBOX STORE → BUSINESS STORE → ACK

1. Broker delivers message (at-least-once)
2. Consumer checks inbox for message_id
3. If not found, process & record message_id in same transaction
4. Apply business update (e.g., ledger, inventory, notification)
5. Acknowledge message safely

If found in inbox → skip processing → ack safely

CONSUMER-SIDE ATOMIC WORKFLOW

- Read message from broker
- Begin local transaction
- Check inbox for message_id
- If absent: apply business update
- Insert message_id into inbox
- Commit transaction and ack message

ATOMIC TRANSACTION BOUNDARY

Inbox Store (Processed IDs) + Business Store (Ledger / State)

Both succeed → commit
Any failure → rollback

Exactly-once business effect

EXACTLY-ONCE EFFECT vs EXACTLY-ONCE DELIVERY

EXACTLY-ONCE EFFECT (PRACTICAL GOAL)

- Message may be delivered multiple times.
- Business side effect happens only once.
- Achieved using inbox pattern + idempotent processing.
- Robust, scalable and widely used in practice.

EXACTLY-ONCE DELIVERY (THEORETICAL)

- Message delivered once and only once.
- Requires complex coordination across the entire pipeline.
- Expensive and hard to achieve end-to-end.
- Not always necessary for correct business outcomes.

FAILURE BEHAVIOR

Crash before commit Message will be retried. No side effects applied.	Safe to retry. Nothing was recorded in inbox.
Commit succeeds but ack fails Message will be redelivered.	Inbox has record. Duplicate will be skipped. Side effect not repeated.
Poison / bad message Processing fails repeatedly. Cannot be processed.	After max retries, send to Dead Letter Queue for manual handling.
Consumer restart In-memory dedup lost. Inbox data is durable.	Previously processed IDs still exist in inbox. Duplicates are skipped.

TRADE-OFFS

- Inbox pattern improves correctness and reliability. Adds storage and write cost.
- High throughput requires retention / archival policy for old inbox records.
- Correctness depends on proper key design (message_id / business id).
- Transaction boundary must include both business update and inbox insert.
- Monitoring and metrics are essential for visibility.

INBOX STORE EXAMPLE

message_id	processed_at
pay_1001	2025-05-15 10:00:01
pay_1002	2025-05-15 10:00:02
pay_1003	2025-05-15 10:00:03
pay_1004	2025-05-15 10:00:04
...	...

Use TTL / archival policy to remove old records safely.

DAY-25 SAMPLE: DURABLE INBOX PAYMENT CONSUMER

KEY COMPONENTS

- **Message** Immutable event with message_id, amount, etc.
- **InboxStore** Durable store for processed message IDs.
- **LedgerStore** Business state (e.g. account balances).
- **InboxConsumer** Atomic handler that applies update and records ID.

END-TO-END FLOW

1. Broker delivers message
2. Check inbox for message_id
3. If absent: apply ledger update + insert inbox record
4. Commit transaction
5. Ack message
6. Update metrics

METRICS EXAMPLE

Applied (unique)	Duplicates Skipped	Duplicate Rate
128	47	26.9% (47 / 175)

NEXT IMPROVEMENTS

- Add TTL / archival policy for inbox records
- Use DB unique constraint on message_id
- Add retry backoff and DLQ routing
- Add partition-aware consumer groups and ordering guarantees
- Add tracing IDs for end-to-end duplicate diagnosis

- Inbox pattern is consumer-side complement to outbox.
- Durable deduplication survives restarts and retries.
- Exactly-once effect is the practical reliability target.
- Transaction boundary must include both updates.
- Metrics + DLQ policy are critical for production.

SAFE TO REPEAT.
CORRECT BY DESIGN.
TRUST IN EVERY RETRY.

20 DAYS OF SYSTEM DESIGN CHALLENGE

Day 25: Inbox Pattern, Exactly-Once Effect, and Consumer Reliability

Day 25 focuses on how consumers achieve exactly-once business effect over at-least-once delivery by combining idempotent processing with a durable inbox record.

It covers:

- inbox pattern fundamentals
- durable deduplication at consumer boundary
- atomic consume + apply workflow
- retry-safe processing semantics
- poison message handling strategy
- operational trade-offs for reliability

In short, Day 25 is about making each logical message affect business state once, even when delivery is repeated.

Core ideas

Why Inbox pattern exists

- Message brokers often provide at-least-once delivery, not exactly-once processing.
- In-memory dedup can fail on restarts and lose seen-message history.
- Inbox pattern stores processed message IDs durably at consumer side.

Consumer-side atomicity

- Consumer reads message and checks inbox table/store for message_id.
- If absent, it applies business update and records inbox entry in one transaction.
- If present, it skips side effects and acknowledges safely.

Exactly-once effect vs exactly-once delivery

- Delivery may happen multiple times.
- Inbox guarantees side effects are applied once per logical message.
- This is usually the practical target in distributed systems.

Failure behavior

- If processing crashes before commit, message can be retried safely.
- If commit succeeds but ack fails, redelivery is skipped due to inbox record.
- Poison messages should be routed to DLQ after bounded retries.

Trade-offs

- Inbox improves correctness with additional storage and write cost.
- Large throughput systems need retention policies for inbox entries.
- Key design and transactional boundaries are critical for correctness.

Day-25 sample: Durable Inbox Payment Consumer

This repository includes a small inbox-pattern sample in both Python and Java.

Functional requirements

- Consume payment events with unique message_id
- Apply account ledger update once per unique message
- Persist processed IDs in durable inbox store
- Skip duplicates safely across repeated deliveries
- Expose processing metrics (applied, duplicate)

High-level components

- Message: event envelope
- InboxStore: durable processed-message registry
- LedgerStore: business side-effect state
- InboxConsumer: atomic process-and-record handler

Data flow

1. Broker delivers message
2. Consumer checks inbox for message_id
3. If unseen, consumer applies ledger update and records inbox entry
4. If seen, consumer skips ledger write
5. Metrics show applied vs duplicate counts

Project structure

```
Day-25/
  README.md
  inbox-pattern-sample/
    python/
      inbox_consumer.py
      demo.py
    java/
      InboxConsumerSystem.java
      Main.java
```

Run the sample

Python

```
cd inbox-pattern-sample\python
```

```
python demo.py
```

Java

```
cd inbox-pattern-sample\java
javac Main.java InboxConsumerSystem.java
java Main
```

Interview takeaways

- Inbox pattern is a consumer-side complement to producer outbox
- Durable deduplication survives consumer restarts and retries
- Exactly-once effect is usually more realistic than exactly-once delivery
- Transaction boundaries must include both business update and inbox insert
- Metrics and DLQ policy are necessary for production reliability

Next improvements

- Add TTL/archival policy for old inbox records
- Use database unique constraints for message IDs
- Add retry backoff with DLQ routing simulation
- Add partition-aware consumer groups and ordering guarantees
- Add tracing IDs for end-to-end duplicate diagnosis

Code Appendix

Python and Java source used in Day 25 sample.

inbox-pattern-sample\python\inbox_consumer.py

```python
from __future__ import annotations

from dataclasses import dataclass


@dataclass(frozen=True)
class Message:
    message_id: str
    account_id: str
    amount_cents: int


class InboxStore:
    """Durable inbox simulation. In production this would be a DB table with unique message_id."""

    def __init__(self) -> None:
        self._processed: set[str] = set()

    def exists(self, message_id: str) -> bool:
        return message_id in self._processed

    def add(self, message_id: str) -> None:
        self._processed.add(message_id)


class LedgerStore:
    def __init__(self) -> None:
        self._balances: dict[str, int] = {}

    def apply(self, account_id: str, amount_cents: int) -> None:
        self._balances[account_id] = self._balances.get(account_id, 0) + amount_cents

    def balance(self, account_id: str) -> int:
        return self._balances.get(account_id, 0)


class InboxConsumer:
    def __init__(self, inbox: InboxStore, ledger: LedgerStore) -> None:
        self._inbox = inbox
        self._ledger = ledger
        self.applied_count = 0
        self.duplicate_count = 0

    def process(self, message: Message) -> str:
        # Simulate atomic business update + inbox insert.
        if self._inbox.exists(message.message_id):
            self.duplicate_count += 1
            return f"duplicate skipped {message.message_id}"

        self._ledger.apply(message.account_id, message.amount_cents)
        self._inbox.add(message.message_id)
        self.applied_count += 1
        return f"applied {message.message_id} amount={message.amount_cents}"

    def balance(self, account_id: str) -> int:
        return self._ledger.balance(account_id)
```

inbox-pattern-sample\python\demo.py

```python
from inbox_consumer import InboxConsumer, InboxStore, LedgerStore, Message


def main() -> None:
    inbox = InboxStore()
```

```
    ledger = LedgerStore()
    consumer = InboxConsumer(inbox, ledger)

    deliveries = [
        Message("m-100", "acct-A", 1200),
        Message("m-101", "acct-A", 800),
        Message("m-100", "acct-A", 1200),  # duplicate replay
        Message("m-102", "acct-A", -300),
        Message("m-103", "acct-B", 900),
        Message("m-101", "acct-A", 800),   # duplicate replay
    ]

    for message in deliveries:
        print(consumer.process(message))

    print("acct-A balance:", consumer.balance("acct-A"))
    print("acct-B balance:", consumer.balance("acct-B"))
    print("applied_count:", consumer.applied_count)
    print("duplicate_count:", consumer.duplicate_count)


if __name__ == "__main__":
    main()
```

inbox-pattern-sample\java\InboxConsumerSystem.java

```
import java.util.HashMap;
import java.util.HashSet;
import java.util.Map;
import java.util.Set;

public class InboxConsumerSystem {
    public static final class Message {
        private final String messageId;
        private final String accountId;
        private final int amountCents;

        public Message(String messageId, String accountId, int amountCents) {
            this.messageId = messageId;
            this.accountId = accountId;
            this.amountCents = amountCents;
        }

        public String getMessageId() {
            return messageId;
        }

        public String getAccountId() {
            return accountId;
        }

        public int getAmountCents() {
            return amountCents;
        }
    }

    public static final class InboxStore {
        private final Set<String> processed = new HashSet<>();

        public boolean exists(String messageId) {
            return processed.contains(messageId);
        }

        public void add(String messageId) {
            processed.add(messageId);
        }
    }

    public static final class LedgerStore {
        private final Map<String, Integer> balances = new HashMap<>();
```

```
        public void apply(String accountId, int amountCents) {
            balances.put(accountId, balances.getOrDefault(accountId, 0) + amountCents);
        }

        public int balance(String accountId) {
            return balances.getOrDefault(accountId, 0);
        }
    }

    public static final class InboxConsumer {
        private final InboxStore inbox;
        private final LedgerStore ledger;
        private int appliedCount = 0;
        private int duplicateCount = 0;

        public InboxConsumer(InboxStore inbox, LedgerStore ledger) {
            this.inbox = inbox;
            this.ledger = ledger;
        }

        public String process(Message message) {
            if (inbox.exists(message.getMessageId())) {
                duplicateCount += 1;
                return "duplicate skipped " + message.getMessageId();
            }

            ledger.apply(message.getAccountId(), message.getAmountCents());
            inbox.add(message.getMessageId());
            appliedCount += 1;
            return "applied " + message.getMessageId() + " amount=" + message.getAmountCents();
        }

        public int balance(String accountId) {
            return ledger.balance(accountId);
        }

        public int getAppliedCount() {
            return appliedCount;
        }

        public int getDuplicateCount() {
            return duplicateCount;
        }
    }
}
```

inbox-pattern-sample\java\Main.java

```
import java.util.List;

public class Main {
    public static void main(String[] args) {
        InboxConsumerSystem.InboxStore inbox = new InboxConsumerSystem.InboxStore();
        InboxConsumerSystem.LedgerStore ledger = new InboxConsumerSystem.LedgerStore();
        InboxConsumerSystem.InboxConsumer consumer = new InboxConsumerSystem.InboxConsumer(inbox, ledger);

        List<InboxConsumerSystem.Message> deliveries = List.of(
            new InboxConsumerSystem.Message("m-100", "acct-A", 1200),
            new InboxConsumerSystem.Message("m-101", "acct-A", 800),
            new InboxConsumerSystem.Message("m-100", "acct-A", 1200),
            new InboxConsumerSystem.Message("m-102", "acct-A", -300),
            new InboxConsumerSystem.Message("m-103", "acct-B", 900),
            new InboxConsumerSystem.Message("m-101", "acct-A", 800)
        );

        for (InboxConsumerSystem.Message message : deliveries) {
            System.out.println(consumer.process(message));
        }

        System.out.println("acct-A balance: " + consumer.balance("acct-A"));
        System.out.println("acct-B balance: " + consumer.balance("acct-B"));
```

```
        System.out.println("applied_count: " + consumer.getAppliedCount());
        System.out.println("duplicate_count: " + consumer.getDuplicateCount());
    }
}
```